

5. BÚSQUEDA LOCAL

Motivación

Existen problemas, como el **Problema de las 8 Reinas**, donde lo único que verdaderamente importa es encontrar el estado objetivo, no el camino hacia el mismo. En este problema, el **objetivo es ubicar 8 reinas en un tablero de ajedrez sin que se ataquen**.

- Bajo la **Formulación I** (matriz 8x8 donde se colocan reinas), el tamaño del espacio de estados es **enorme** ($1,8 \times 10^{14}$).
- Con la **Formulación II*** (tupla donde no se atacan diagonalmente), el número de estados se reduce drásticamente a **2057**. En muchos problemas de búsqueda, **si conocemos un estado objetivo, es trivial encontrar una solución** (un camino desde el estado inicial a él).

💡 Nuevos algoritmos de búsqueda: Búsqueda local

Para solventar el alto consumo de memoria, surgen nuevos algoritmos que **no almacenan el padre, ni los ancestros, ni la frontera**.

- Una **búsqueda local** mantiene unos **pocos estados en memoria** y, en cada iteración, **actualiza el estado actual por alguno de sus sucesores**, eligiéndolo y deteniéndose bajo ciertos criterios.
- **Características generales:** Tienen un **muy bajo consumo de memoria**, pueden encontrar **soluciones razonablemente buenas en espacios muy grandes o infinitos**, pero **no garantizan completitud ni optimalidad**, pueden ciclar y se pueden atascar en estados terminales.
- **Formulación de estados completa:** La búsqueda local **no es compatible con formulaciones incrementales** (no se puede progresar al llegar a un estado no-objetivo terminal). Se usan formulaciones donde **todas las componentes ya están ubicadas** y las acciones simplemente las mueven o intercambian. De esta forma **no hay estados terminales**.
- **Costos de camino:** Son **irrelevantes**, ya que se sigue un único camino sin registrar alternativas inexploradas.
- **Función objetivo:** Es una función f que **asigna a cada estado un valor numérico (valor objetivo)**. La búsqueda local elige los sucesores con el fin de **maximizar (o minimizar)** este valor, buscando que alcance su extremo en los estados objetivos del problema.
- **Paisaje del espacio de estados:** Se explora un paisaje evaluando y modificando estados, subiendo montañas (si se busca maximizar) o bajando valles (si se busca minimizar).

Búsqueda de ascensión de colinas (Hill-climbing search)

Es la búsqueda local más simple (búsqueda local avara). Supongamos que buscamos maximizar: en cada iteración, el algoritmo **se mueve al sucesor con mayor valor objetivo, siempre que mejore el valor objetivo actual**, hasta atascarse.

- **Características:** Tiene **muy bajo consumo de memoria** y **nunca cicla**, pero **se atasca en máximos (locales o globales)** y **no garantiza completitud ni optimalidad**. ¡Nunca se mueve a un sucesor que no mejore el valor actual!.

Máximos locales y globales

El algoritmo de ascensión de colinas realiza un rápido ascenso, pero **al llegar a una cima se atasca**, la cual puede ser un **máximo global o un máximo local**.

- **Caminos redundantes y mesetas:** El algoritmo se atasca en picos, máximos locales planos y terrazas.

- **Limitando movimientos laterales:** Para escapar de mesetas, se pueden permitir movimientos hacia estados con un **valor objetivo igual al actual**. Sin embargo, **permitir infinitos movimientos laterales consecutivos no garantiza escapar**, ya que se podría estar ciclando o en un máximo local plano.
- Con movimientos laterales combinados con **reinicio aleatorio**, se pueden resolver problemas inmensos (como millones de reinas) en fracciones de minuto.

Búsqueda Tabú (Tabu Search)

Es una búsqueda local que incorpora mejoras a la ascensión de colinas para **escapar de máximos locales que no son globales**. **Siempre se mueve al sucesor con mejor valor objetivo, ya sea mejor, peor o igual que el actual**.

- **Lista tabú:** Para mitigar el riesgo de ciclar, usa una **memoria de corto plazo** que restringe su movimiento guardando información de iteraciones pasadas.
- **¿Qué se almacena?:** 1) **Acciones** (que podrían revertir las recientes), 2) **Estados** (restringe menos pero consume **más tiempo y memoria**) o 3) **Propiedades** (atributos de estados o acciones).
- **¿Por cuánto tiempo?:** Mantener los elementos para siempre consumiría memoria excesiva y aumentaría la probabilidad de atascarse. Se usa una **Capacidad limitada** (la lista funciona como una cola, borrando lo más viejo) o un **Tenor de tabú** (se borran los elementos que superan un límite de iteraciones establecido).
- **Criterio de parada:** Como permite empeorar el estado y podría no terminar, requiere un criterio de parada: **iteraciones totales/tiempo**, **iteraciones sin mejoras**, o al **sobrepasar un valor umbral**.
- **Componentes adicionales:**
 - Criterio de aspiración:** Permite **ignorar tabúes si la acción lleva a un estado con un valor objetivo mayor** al del mejor estado encontrado hasta el momento.
 - Diversificación:** Mecanismo con **memoria de largo plazo** que fuerza la búsqueda en regiones inexploradas (usando la **memoria de frecuencia**) mediante estrategias de **reinicio o continuas**.

Seteo de parámetros

Las configuraciones exactas de qué almacenar, por cuánto tiempo, o cuándo parar **se determinan experimentalmente**. Los algoritmos tabú **son extremadamente sensibles al seteo de sus parámetros**, cambiando su performance drásticamente.

Performance de BÚSQUEDA-TABÚ

- Tiene un **bajo consumo de memoria**, aunque levemente mayor que la ascensión de colinas.
- **No se atasca en máximos locales** porque puede moverse a peores valores.
- **Reduce el riesgo de ciclar** gracias a la lista tabú (aunque no lo elimina).
- Garantiza terminación mediante un **criterio de parada**, pero en general **no garantiza completitud ni optimalidad**.

Búsqueda local en espacios continuos

En el mundo real (longitud, masa, tiempo), los estados tienen **infinitos sucesores**, imposibilitando los algoritmos vistos. Ejemplo clásico: Problema de ubicación de aeropuerto, buscando que la suma de las distancias al cuadrado sea mínima.

Discretizar vs. no discretizar

Hay dos formas de abordar espacios continuos:

1. **Discretizar:** Consiste en restringir el problema para que el conjunto de **acciones posibles sea finito** (por ejemplo, mover la coordenada en una cantidad fija $\pm\delta$). Luego se resuelve con cualquier algoritmo local ya visto.
2. **No discretizar:** En el espacio continuo absoluto se usan **conceptos de cálculo matemático**. Específicamente, se analiza el campo escalar utilizando el **Gradiente** (∇f) compuesto por las derivadas parciales de cada variable.

6. Problemas de Satisfacción de Restricciones (CSPs)

Más definiciones de CSPs

Un Problema de Satisfacción de Restricciones (CSP) se define por un conjunto de **variables**, **dominios** y **restricciones**.

- **Soluciones:** Una **asignación asocia un valor a algunas o todas las variables**. Una **solución es una asignación consistente y completa** (no siempre es única).

Clasificación de variables

Según su dominio, una variable se clasifica en:

- **Discreta finita:** Tiene un dominio con una cantidad finita de valores. El estudio de la unidad **se limita a este tipo de variables**.
- **Discreta infinita:** Por ejemplo, los números Naturales.
- **Continua:** Por ejemplo, los números Reales.

Clasificación de restricciones

- **Unaria:** Restringe el valor de una **única variable**. Observación 1: **Toda restricción unaria puede eliminarse** modificando el dominio de la variable.
- **Binaria:** Relaciona **dos variables**.
- **Global:** Relaciona un **número arbitrario de variables**. Observación 2: **Toda restricción global puede transformarse en un conjunto de restricciones binarias**.
- Observación 3: **Todo CSP se puede transformar en un CSP equivalente que tiene únicamente restricciones binarias**.

Grafo de restricciones

Los CSP se visualizan con grafos donde **los nodos son las variables y los arcos conectan a los pares de variables que participan en una restricción binaria**.

Búsqueda clásica / Formulación para CSPs

Podemos convertir un CSP en un problema de búsqueda.

- **Estados:** Son todas las **asignaciones consistentes** (parciales o completas).
- **Estado inicial:** Asignación vacía $\{\}$.
- **Acciones:** Asignar a una variable no asignada un valor de su dominio de modo que **mantenga la asignación consistente**.
- **Modelo de transiciones:** Integra la variable y su valor a la asignación parcial actual.
- **Test objetivo:** ¿La asignación **es completa**?
- **Costo de camino:** Es la suma de los costos individuales, donde **cada acción tiene un costo de 1**.

- **Tamaño del árbol de búsqueda:** El número de asignaciones completas está dado por **el producto de los cardinales de los dominios**.

Algoritmo de búsqueda vuelta atrás (Backtracking)

- Es una implementación **recursiva** que elige un valor consistente para una variable, y se repite hasta hallar una asignación completa.
- Si a una variable **no le quedan valores consistentes, devuelve un fallo** y la búsqueda **retrocede a la última llamada abierta**.
- Usa implícitamente el mismo estado inicial, acciones y test objetivo para cualquier problema; es decir, **no necesita funciones específicas del problema a resolver**.
- **Garantiza la completitud** siempre y cuando los dominios sean finitos.

Búsqueda local

Otra alternativa para resolver un CSP es mediante la búsqueda local, que requiere:

1. Formulación de estados completa:

- Los estados son las **asignaciones completas (ya sean consistentes o inconsistentes)**.
- Las **acciones representan reasignar** a una variable otro valor de su dominio.
- El test objetivo valida si el estado es consistente.

2. Función objetivo:

- La función f **se define como el número de restricciones violadas** por una asignación.
- El **objetivo es minimizar f** (o maximizar $-f$); siendo 0 el mínimo global que indica que se halló una solución.

Backtracking mejorado

Los algoritmos de backtracking requieren **criterios (estáticos o dinámicos)** para elegir la próxima variable a asignar y para ordenar sus valores. Estas **buenas decisiones pueden evitar retrocesos futuros** y acelerar la búsqueda independientemente del problema específico.

3. Comprobación hacia adelante (Forward checking)

- **Regla:** Siempre que se asigna una variable X_i , se revisan las variables no asignadas X_j relacionadas y **se quitan de su dominio los valores que sean inconsistentes** con la nueva asignación.
- Si alguna variable no asignada **se queda con dominio vacío, la asignación es inconsistente y se debe retroceder**.
- **Ventajas:** Es fácil de incorporar, **reduce el tamaño de los dominios (y el tiempo de backtracking)**, y detecta anticipadamente algunas inconsistencias.
- **Limitaciones:** Sólo restringe los dominios de las variables directamente relacionadas a la recién asignada; es decir, **no mira lo suficientemente adelante** entre variables que aún no han sido asignadas.

Arco consistencia

- Decimos que una variable X_i es **arco-consistente** con X_j si **para todo valor en el dominio de X_i , existe algún valor en el dominio de X_j que cumple la restricción** que las relaciona.
- Es posible **forzar la arco-consistencia** eliminando del dominio de X_i todos los valores que no cumplan la relación.

4. Propagación de restricciones

- La comprobación hacia adelante y la arco-consistencia son formas de inferencia llamadas **propagación de restricciones: usar las restricciones para descartar valores inconsistentes**.
- Restringir el dominio de una variable permite restringir el de otras, generando una **cadena**.

- El **Mantenimiento de la Consistencia de Arco (MCA o AC-3)** impone la arco-consistencia iterativamente hasta que no hay más cambios en los dominios, siendo un **algoritmo estrictamente más potente** que la comprobación hacia adelante.

Restricciones globales – Alldiff y Atmost

- **Alldiff (Todas distintas): Test de inconsistencia:** Si hay n posibles valores distintos para m variables, y $n < m$, **la restricción no se puede verificar**.
- **Atmost (A lo sumo): Test de inconsistencia:** Si la suma del **menor valor de cada dominio** de las variables es **mayor al tope K** , el CSP no tiene solución.

Estructura de los problemas y Teoría de grafos

- La dificultad para resolver un CSP está ligada a la **estructura del grafo de restricciones**.
- **Componente conexa:** Es un subgrafo inducido que es **conexo** (todo par de vértices tiene un camino) y **maximal** (se desconecta si se agrega otro vértice).
- Cada componente conexa corresponde a un **subproblema independiente**.
- **Ventajas:** Dividir un problema en k subproblemas independientes reduce la complejidad computacional masivamente, pasando de tener una cantidad de asignaciones de d^n a $k \cdot d^{n/k}$.

Aquí tienes la información de la Unidad 7 condensada detalladamente, manteniendo los subtítulos, destacando lo más importante en negrita y con sus respectivas citas:

7. Búsquedas inspiradas en la naturaleza

La naturaleza como fuente de inspiración

Estos algoritmos buscan **simular cómo la naturaleza resuelve ciertos problemas** basándose en fenómenos biológicos, leyes físicas y procesos químicos. Se trata de una familia muy amplia de algoritmos que **han demostrado ser muy efectivos en problemas de optimización** y continúan siendo un área muy productiva en el diseño algorítmico. Gran parte de su éxito se debe a que incorporan **mecánicas estocásticas** (probabilísticas).

Recocido Simulado (Simulated Annealing - SA)

El recocido simulado es un algoritmo de **búsqueda local que utiliza un mecanismo probabilístico, regulado por un parámetro de “temperatura”, para escapar de máximos locales**. Se inspira en el tratamiento térmico (recocido) donde un material se calienta y se enfría progresivamente para mejorar sus propiedades. En el algoritmo, **los “malos” movimientos son más probables al comienzo y se vuelven menos probables en las últimas iteraciones** conforme la temperatura baja.

- **Esquema de recocido o enfriamiento:** Es una función que determina la temperatura en función del número de iteraciones. Los tipos de enfriamiento más comunes son el Lineal, el Logarítmico y el **Geométrico o Exponencial (este último es el más utilizado)**. El algoritmo se detiene típicamente cuando la temperatura se acerca a cero.
- **Función de probabilidad de aceptación:** Determina la probabilidad de aceptar un estado sucesor. Si el sucesor es mejor, se acepta siempre; si es peor, **la probabilidad de aceptarlo depende de la temperatura y de qué tan malo sea el sucesor**. Típicamente se usa la función exponencial:

$$P(s, s', t) = e^{(f(s') - f(s))/t}$$
Fijada una temperatura y un estado, **la probabilidad de aceptar un sucesor es menor mientras peor sea este**.

Algoritmo Genético (Genetic Algorithm - GA)

Los algoritmos genéticos son búsquedas basadas en poblaciones que se inspiran en la teoría evolutiva de la **selección natural**. Constan de varios pasos en su ciclo:

- **1. Población inicial:** El algoritmo comienza con un conjunto de k individuos generados aleatoriamente, donde el genoma de cada uno se representa usualmente con un *string*.

- **2. Función de fitness (o función objetivo):** Califica a cada individuo, devolviendo **valores más altos para los mejores individuos** (en un problema de maximización).
- **3. Selección:** Consiste en elegir aleatoriamente pares de individuos para reproducirse, donde **la probabilidad de que un individuo sea elegido es directamente proporcional a su valor de fitness**.
- **4. Cruzamiento:** Consiste en generar nuevos individuos combinando la información genética de ambos padres.
- **5. Mutación:** Se realiza con una **probabilidad baja** y consiste en aplicar un **cambio aleatorio en la información genética** del nuevo individuo.

Evolución de las generaciones: Inicialmente, la población es diversa. Sin embargo, a medida que avanzan las iteraciones, **se produce una convergencia de la población (se pierde diversidad genética)**, ya que la selección favorece reproducir individuos de mayor calidad y sus características se propagan. *(Nota: Para problemas como el Problema del Viajante de Comercio o TSP, existen representaciones especiales ordinales o de camino, y operadores de mutación específicos como el movimiento de 2-opt entre arcos, para evitar romper soluciones válidas al cruzar o mutar).*

Optimización por Colonia de Hormigas (Ant Colony Optimization - ACO)

Inspirado en el **rastro de feromonas** que usan las hormigas para comunicarse, es también una búsqueda local basada en población.

- **1. Construcción de soluciones:** Una población de m hormigas artificiales construye soluciones de forma incremental y estocástica. En cada paso, la probabilidad de elegir el siguiente componente **depende de forma directamente proporcional al nivel de feromonas $\tau(i,j)$ e inversamente proporcional a la distancia $dist(i,j)$** .
- **2. Depósito y evaporación:** Las feromonas transmiten la experiencia acumulada hacia el futuro. El **depósito incrementa el nivel de feromonas** para reforzar componentes de buenas soluciones (tours más cortos reciben más feromonas). La **evaporación reduce periódicamente los niveles de feromona** basándose en un factor ρ , lo que sirve para **evitar la convergencia prematura** y favorecer la exploración de soluciones nuevas.

Optimización por Enjambre de Partículas (Particle Swarm Optimization - PSO)

Este algoritmo simula una población que se mueve ordenadamente, inspirándose en bandadas de aves o cardúmenes de peces. Se diferencia en que se utiliza para **optimizar funciones reales no lineales en el espacio $\mathbb{R}^n$** .

- **Componentes:** Cada partícula (individuo) modela una solución del problema y está conformada por una **posición x_i y una velocidad inicial v_i** . Cada individuo posee una **memoria individual ($pbest_i$)** que guarda su mejor posición personal, mientras que el enjambre entero posee una **memoria colectiva ($gbest$)** que guarda la mejor posición global alcanzada.
- **Ecuación de velocidad y posición:** Cada partícula actualiza su comportamiento basándose en tres vectores: su propia trayectoria ponderada por un **factor de inercia (w)**, el acercamiento a su **mejor posición personal ($pbest_i$)**, y el acercamiento a la **mejor posición global ($gbest$)**. El cálculo de la nueva velocidad multiplica la atracción hacia $pbest$ y $gbest$ por factores aleatorios $rand()$ y por unas **constantes de aceleración c_1 y c_2** . Finalmente, la posición de la partícula se actualiza sumando a su posición previa la nueva velocidad.

Resumen

Todos estos algoritmos incorporan elementos de estocasticidad para lograr resolver problemas complejos de optimización. Mientras que **el recocido simulado mueve un solo estado**, **los algoritmos genéticos, las colonias de hormigas y el enjambre de partículas efectúan una búsqueda local basada en poblaciones completas de posibles soluciones**.